CENTRO DE ENSEÑANZA TÉCNICA Y SUPERIOR



LENGUAJES DE PROGRAMACIÓN ORIENTADA A OBJETOS
Profra. Vanessa Miranda

ProyectoFinal_ICC202_2026
BORRADOR REPORTE TECNICO

Presentado por:

Omar Muñoz - 36471
Paul Alvarez - 40683
Karim Herrera - 39219
Abel Martinez - 38531

Tijuana, B.C. a 7 de mayo de 2026

Nombre de proyecto: The Catacombs

Objetivo principal del sistema:

Desarrollar un videojuego tipo dungeon-crawler en C++ utilizando programación orientada a objetos, donde el jugador pueda participar en combates por turnos, usar movimientos y pociones, ganar experiencia, subir de nivel y obtener reliquias con diferentes rarezas. El sistema busca integrar clases relacionadas entre sí para representar de forma organizada la lógica del juego, la progresión del jugador y la interacción con enemigos.

Descripción breve del problema que resuelve:

El proyecto resuelve el reto de crear un sistema interactivo usando conceptos de programación orientada a objetos dentro de un contexto funcional y dinámico. En vez de manejar clases separadas sin conexión, The Catacombs organiza el juego en módulos como jugador, enemigos, movimientos, reliquias, pociones y el motor de combate. Gracias a esto, cada parte cumple una función específica y el sistema puede expandirse de manera más ordenada sin cargar toda la lógica en un solo archivo como el main.cpp.

Además, la solución permite representar mecánicas típicas de un videojuego de combate, como el cálculo de daño, efectos especiales de movimientos, recompensas después de cada pelea, uso de objetos y relaciones de herencia entre entidades. También se aplican comportamientos polimórficos mediante las reliquias, haciendo que diferentes objetos modifiquen la experiencia del jugador de distintas formas.

Restricciones o limitaciones conocidas:

El sistema actual ya cuenta con combate por turnos, clases base, movimientos, pociones, reliquias, RelicPool, experiencia, subida de nivel e interfaz gráfica básica con SFML. Sin embargo, todavía existen funciones que deben reforzarse o integrarse completamente para evitar errores de ejecución o uso inesperado por parte del usuario.

Entre las partes pendientes se encuentra el sistema de pisos y puertas, el cual aún no ha sido implementado completamente como un módulo funcional. De igual forma, las puertas corruptas y algunos efectos relacionados con ProbabilityRelic dependen de futuros sistemas de navegación que todavía no han sido integrados.

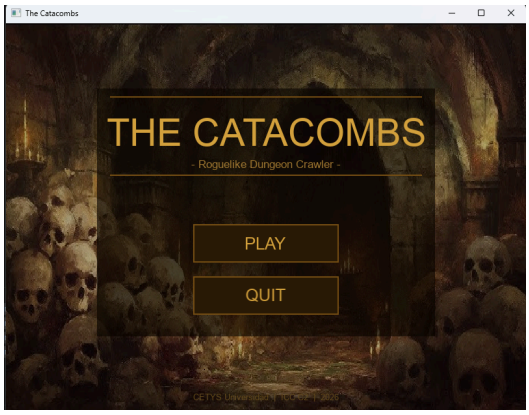
La interfaz gráfica actual cumple con mostrar el combate y la información principal del juego, pero todavía es bastante básica. A futuro se planea agregar selección visual de movimientos mediante botones, además de sprites, animaciones y assets definitivos que hagan la experiencia más dinámica y visualmente completa.

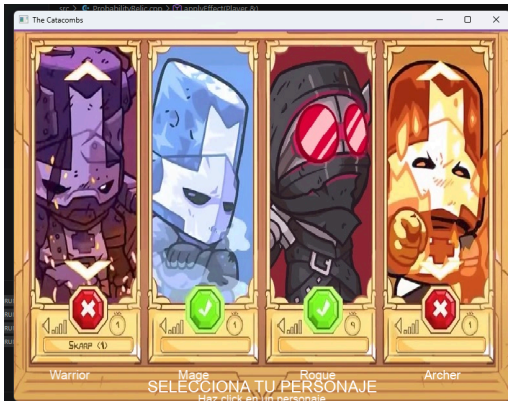
El sistema de pociones ya funciona correctamente, aunque todavía necesita mejoras para permitir al jugador seleccionar específicamente qué poción desea usar durante la partida. También sigue pendiente la implementación del jefe final con múltiples fases, pensado como el combate más complejo del juego.

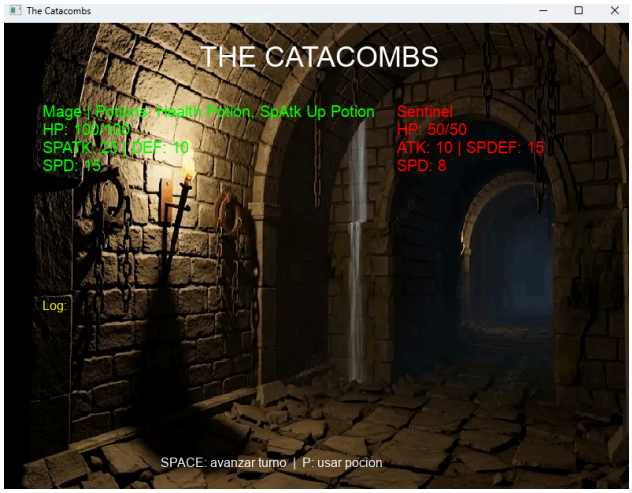
Finalmente, es necesario reforzar las validaciones de entradas del usuario para evitar errores como índices inválidos, selección incorrecta de acciones o uso de objetos inexistentes. Además, algunas mecánicas planeadas, como recompensas especiales, sistemas avanzados de pisos y ciertas sinergias entre reliquias, aún se encuentran en desarrollo y no han sido integradas completamente al proyecto.

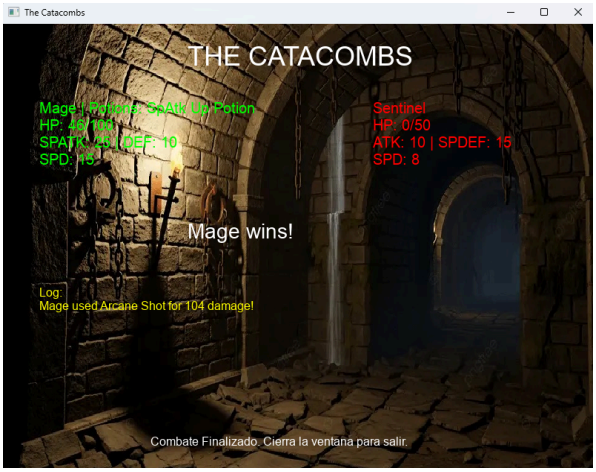
DIAGRAMA

Capturas del programa









Justificación de conceptos de POO:

CONCEPTO	COMO SE APLICO
Herencia	Se utilizó para reutilizar atributos y funciones comunes entre clases relacionadas. Player y Enemy heredan de Entity, mientras que MiniBoss hereda de Enemy. También se aplica en el sistema de reliquias, donde StatRelic, MoveRelic, ProbabilityRelic y UtilityRelic heredan de Relic.
Polimorfismo	Se aplicó mediante métodos virtuales. La clase Relic declara applyEffect(Player&) como método virtual puro, y cada subclase lo redefine para aplicar un efecto diferente. Esto permite guardar distintas reliquias en un mismo vector y ejecutar el efecto correcto.
Encapsulamiento	Los atributos principales se mantienen como private o protected, evitando acceso directo desde fuera de la clase. El acceso se realiza mediante getters, setters o funciones específicas como takeDamage(), heal(), addRelic() y usePotion().
STL	Se utilizaron contenedores de la biblioteca estándar de C++ para manejar colecciones dinámicas. Esto facilita almacenar movimientos, pociones, reliquias y pools sin definir arreglos fijos manualmente.
Abstraccion	Se crearon clases generales que representan conceptos del sistema sin depender de un caso específico. Entity representa una entidad de combate y Relic representa una reliquia genérica. Las clases hijas implementan detalles concretos.

Composición y Agregación	Player contiene objetos que forman parte de su funcionamiento, como movimientos, pociones y reliquias. Entity también contiene movimientos. Esto permite construir objetos más complejos a partir de otros objetos.
Modularidad	El proyecto separa la lógica en archivos .h y .cpp, evitando que todo el sistema dependa de main.cpp. Cada clase tiene una responsabilidad clara dentro del videojuego.
Uso de punteros	Se usan punteros a clase base para manejar objetos derivados de manera polimórfica. Esto es necesario porque Relic es abstracta y no se pueden guardar objetos Relic directamente.